

UNIT-II

DIVIDE-AND-CONQUER

GENERAL METHOD: -

Divide-and-conquer algorithms work according to the following general plan:

1) Divide:

A problem is divided into a number of subproblems (which are smaller instances of the given problem) of the same type, ideally of about equal size.

2) Conquer:

The subproblems are solved (typically recursively). However, if the subproblems are small enough, they are solved in a straightforward manner.

3) Combine:

If necessary, the solutions to the subproblems are combined to get a solution to the original problem.

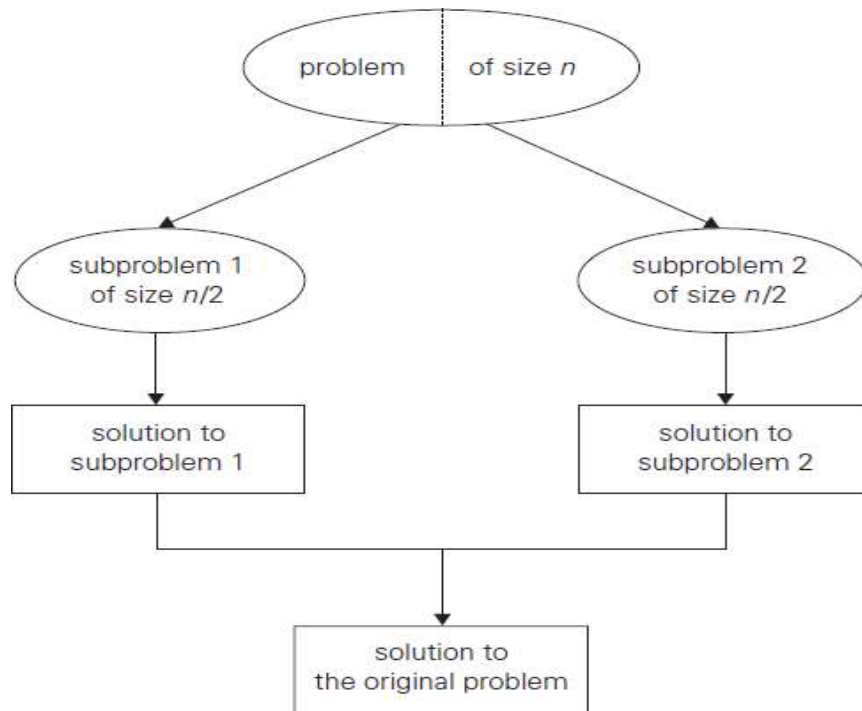


Figure: Divide-and-conquer technique (typical case)

Control abstraction for divide and conquer: -

```
1. Algorithm DandC(P)
2. {
3.   if Small(P) then
4.     return S(P);
5.   else
6.     {
7.       Divide P into smaller instances P1, P2, ..., Pk, where  $k \geq 1$ ;
8.       Apply DandC to each of these k sub-problems;
9.       return Combine(DandC(P1), DandC(P2), ..., DandC(Pk));
10.    }
11.}
```

→ Small(P) is a Boolean-valued function that determines whether the input size is small enough to compute without splitting.

If this is so, the function S(P) is invoked. Otherwise, the problem P is divided into smaller sub-problems.

→ Combine() is a function that determines the solution to P using the solutions to the k sub-problems P1, P2, ..., Pk.

Computing the time complexity of DandC:-

→ If the size of the problem P is n and the sizes of the k sub-problems are $n_1, n_2, \dots, n_k$ respectively, then the computing time of DandC is described by the recurrence relation:

$$T(n) = \begin{cases} g(n), & \text{if } n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n), & \text{otherwise} \end{cases}$$

where,

- $T(n)$ is the time for DandC on any input of size n
- $g(n)$ is the time to compute the answer directly for small inputs
- $f(n)$ is the time for dividing P into sub-problems and combining the solutions of sub-problems.

→ The time complexity of many divide-and-conquer algorithms is given by recurrences of the form:

$$T(n) = \begin{cases} c, & \text{if } n = 1 \\ a T(n/b) + f(n), & \text{if } n > 1 \end{cases}$$

where a, b and c are constants.

We assume that n is a power of b (i.e., $n = b^k$, for some $k \geq 1$).

Eg :-1. Solve the following recurrence relation using substitution method:

$$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ 2 T(n/2) + n, & \text{otherwise} \end{cases}$$

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2[2T(n/4) + n/2] + n = 4T(n/4) + 2n \\ &= 4[2T(n/8) + n/4] + 2n = 8T(n/8) + 3n = 2^3 T(n/2^3) + 3n \end{aligned}$$

After applying this substitution k times,

$$T(n) = 2^k T(n/2^k) + kn$$

To terminate this substitution process, we switch to the closed form $T(1) = 1$, which happens when $2^k = n$ which implies $k = \log_2 n$

$$\text{So, } T(n) = nT(1) + n \log_2 n$$

$$\begin{aligned} T(n) &= n + n \log n \\ &= O(n \log n) \end{aligned}$$

Note:

$$(1) \quad 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

$$(2) \quad 1 + r + r^2 + \dots + r^k = \frac{r^{k+1} - 1}{r - 1}$$

$$(3) \quad 1 + \frac{1}{r} + \frac{1}{r^2} + \dots + \frac{1}{r^k} = \frac{1 - r^{k+1}}{1 - r}$$

$$(4) \quad 1.2 + 2.2^2 + 3.2^3 + \dots + k.2^k = (k - 1)2^{k+1} + 2$$

$$(5) \quad 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$(6) \quad 1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

2)

s) solve

$$T(n) = 2T(n/2) + \log n$$

sol:

$$T(n) = 2T(n/2) + \log n$$

$$= 2(2T(n/2^2) + \log \frac{n}{2}) + \log n = 2^2 \log \frac{n}{2^2} + 2 \log \frac{n}{2} + \log n$$

$$= 2^3 T(n/2^3) + 2^2 \log \frac{n}{2^2} + 2 \log \frac{n}{2} + \log n$$

After k substitutions,

$$T(n) = 2^k T(n/2^k) + \log n + 2 \log \frac{n}{2} + 2^2 \log \frac{n}{2^2} + \dots + 2^{k-1} \log \frac{n}{2^{k-1}}$$

$$= 2^k T\left(\frac{n}{2^k}\right) + \log n + 2(\log n - \log 2) + 2^2(\log n - \log 2^2) + \dots + 2^{k-1}(\log n - \log 2^{k-1})$$

$$= 2^k T\left(\frac{n}{2^k}\right) + \log n + 2 \log n - 2 \cdot 1 + 2^2 \log n - 2 \cdot 2^2 + \dots + 2^{k-1} \log n - (k-1) \cdot \frac{k-1}{2}$$

$$= 2^k T\left(\frac{n}{2^k}\right) + \log n (1 + 2 + 2^2 + \dots + 2^{k-1}) - (1 \cdot 2 + 2 \cdot 2^2 + \dots + (k-1) \cdot \frac{k-1}{2})$$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \log n \left(\frac{k}{2} - 1\right) - \left(\frac{(k-2)^2}{2} + 2\right)$$

Assuming $n = 2^k \Rightarrow k = \log_2 n$

$$\Rightarrow T(n) = n T(1) + \log n (n-1) - \left(\frac{(\log n - 2)^2}{2} + 2\right)$$

$$= nc + 2 \log n - \log n - 2 \log n + 2n - 2$$

$$T(n) = cn + 2n - \log n - 2$$

$$\Rightarrow T(n) = O(n)$$

Solving Recurrence Relations Using Recursion Tree Method: -

(Reference: <https://www.gatevidyalay.com/recursion-tree-solving-recurrence-relations/>)

Problem-1:

Solve the following recurrence relation using recursion tree method:

$$T(n) = 2T(n/2) + n$$

Solution-

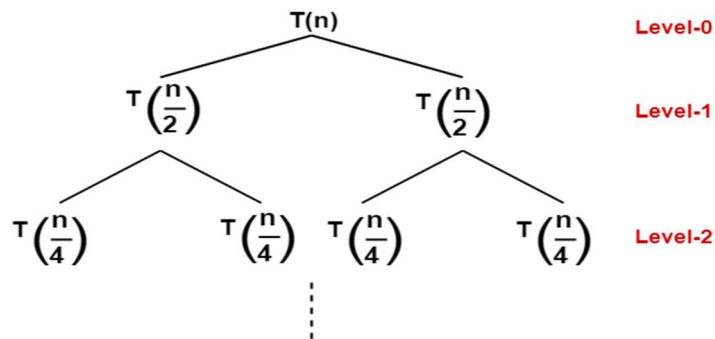
Step-01:

Draw a recursion tree based on the given recurrence relation.

The given recurrence relation shows-

- A problem of size n will get divided into 2 sub-problems of size $n/2$.
- Then, each sub-problem of size $n/2$ will get divided into 2 sub-problems of size $n/4$ and so on.
- At the bottom most level, the size of sub-problems will reduce to 1.

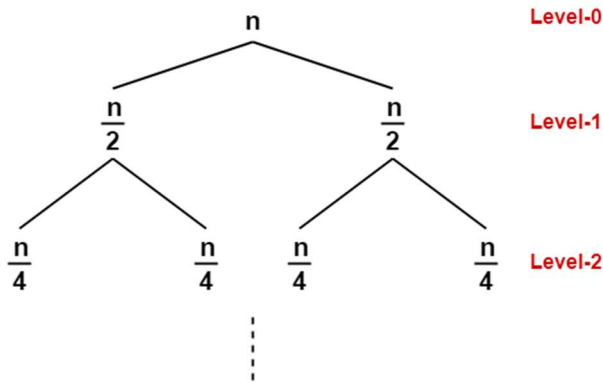
This is illustrated through following recursion tree-



The given recurrence relation shows-

- The cost of dividing a problem of size n into its two sub-problems and then combining their solutions is n .
- The cost of dividing a problem of size $n/2$ into its two sub-problems and then combining their solutions is $n/2$ and so on.

This is illustrated through following recursion tree where each node represents the cost of the corresponding sub-problem:



Step-02:

Determine cost of each level:

- Cost of level-0 = n
- Cost of level-1 = $n/2 + n/2 = n$
- Cost of level-2 = $n/4 + n/4 + n/4 + n/4 = n$, and so on.

Step-03:

Determine total number of levels in the recursion tree:

- Size of problem at level-0 = $n/2^0$
- Size of each sub-problem at level-1 = $n/2^1$
- Size of each sub-problem at level-2 = $n/2^2$

Continuing in similar manner, we have:

Size of each sub-problem at level- i = $n/2^i$

Suppose at level- x (last level), size of each sub-problem becomes 1. Then

$$n / 2^x = 1$$

$$2^x = n$$

Taking log on both sides, we get,

$$x = \log_2 n$$

$$\therefore \text{Total number of levels in the recursion tree} = x+1 = \log_2 n + 1$$

Step-04:

Determine number of nodes in the last level:

- Level-0 has 2^0 nodes i.e., 1 node
- Level-1 has 2^1 nodes i.e., 2 nodes
- Level-2 has 2^2 nodes i.e., 4 nodes

Continuing in similar manner, we have,

Last level (i.e., Level - $\log_2 n$) has $2^{\log_2 n} = n^{\log_2 2} = n$ nodes.

Step-05:

Determine cost of last level:

Cost of last level = $n * T(1) = cn = \theta(n)$.

Step-06:

Add costs of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation:

$$T(n) = \underbrace{\{ n + n + n + \dots \}}_{\text{For } \log_2 n \text{ levels}} + \theta(n)$$

$$= n \log_2 n + \theta(n)$$

$$= \theta(n \log n)$$

Problem-2:

Solve the following recurrence relation using recursion tree method:

$$T(n) = T(n/5) + T(4n/5) + n$$

Solution-

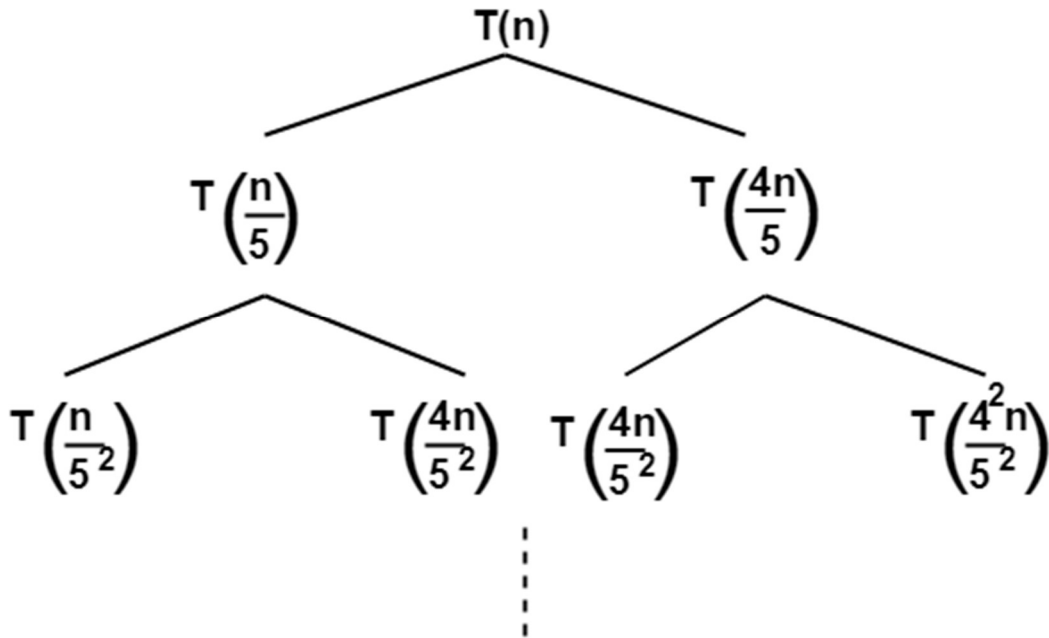
Step-01:

Draw a recursion tree based on the given recurrence relation:

The given recurrence relation shows-

- A problem of size n will get divided into two sub-problems- one of size $n/5$ and another of size $4n/5$.
- Then, sub-problem of size $n/5$ will get divided into two sub-problems- one of size $n/5^2$ and another of size $4n/5^2$.
- On the other side, sub-problem of size $4n/5$ will get divided into two sub-problems- one of size $4n/5^2$ and another of size $4^2n/5^2$, and so on.
- At the bottom most level, the size of sub-problems will reduce to 1.

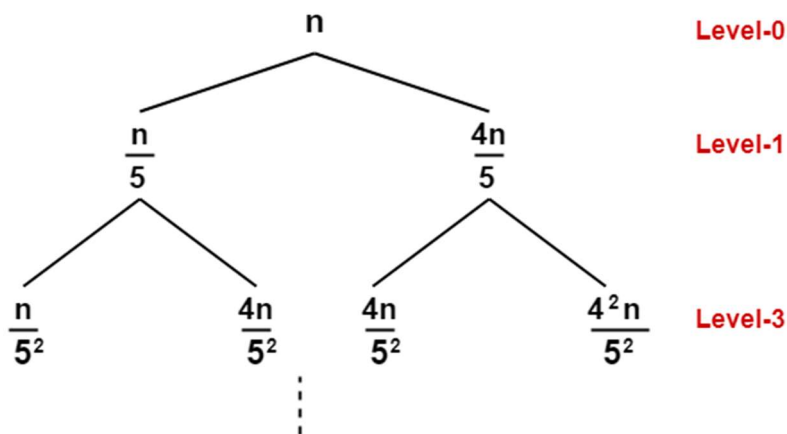
This is illustrated through following recursion tree:



The given recurrence relation shows-

- The cost of dividing a problem of size n into its 2 sub-problems and then combining its solution is n .
- The cost of dividing a problem of size $n/5$ into its 2 sub-problems and then combining its solution is $n/5$.
- The cost of dividing a problem of size $4n/5$ into its 2 sub-problems and then combining its solution is $4n/5$ and so on.

This is illustrated through following recursion tree where each node represents the cost of the corresponding sub-problem:



Step-02:

Determine cost of each level-

- Cost of level-0 = n
- Cost of level-1 = $n/5 + 4n/5 = n$
- Cost of level-2 = $n/5^2 + 4n/5^2 + 4n/5^2 + 4^2n/5^2 = n$

Step-03:

Determine total number of levels in the recursion tree:

We will consider the rightmost subtree (i.e., largest subproblem) as it goes down to the deepest level,

- Size of problem at level-0 = $(4/5)^0 n$
- Size of larger sub-problem at level-1 = $(4/5)^1 n$
- Size of larger sub-problem at level-2 = $(4/5)^2 n$

Continuing in similar manner, we have,

Size of larger sub-problem at level- i = $(4/5)^i n$

Suppose at level- x (last level), size of larger sub-problem becomes 1. Then,

$$(4/5)^x n = 1$$

$$n = (5/4)^x$$

Taking log on both sides, we get,

$$x = \log_{5/4} n$$

$$\therefore \text{Total number of levels in the recursion tree} = x+1 = \log_{5/4} n + 1$$

Step-04:

Determine number of nodes in the last level:

- Level-0 has 2^0 nodes i.e., 1 node
- Level-1 has 2^1 nodes i.e., 2 nodes
- Level-2 has 2^2 nodes i.e., 4 nodes

Continuing in similar manner, we have,

Last level (i.e., Level- x =Level - $\log_{5/4} n$) has $2^{\log_{5/4} n}$ nodes = $n^{\log_{5/4} 2}$.

Cost of each node in the last level is $T(1)$.

Step-05:

Determine cost of last level:

$$\text{Cost of last level} = n^{\log_{5/4} 2} * T(1) = \theta(n^{\log_{5/4} 2})$$

Step-06:

Add costs of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation:

$$T(n) = \underbrace{\{ n + n + n + \dots \}}_{\text{For } \log_{5/4} n \text{ levels}} + \theta(n^{\log_{5/4} 2})$$

For $\log_{5/4} n$ levels

$$\begin{aligned} &= n \log_{5/4} n + \theta(n^{\log_{5/4} 2}) \\ &= \theta(n \log_{5/4} n). \end{aligned}$$

Problem-3:

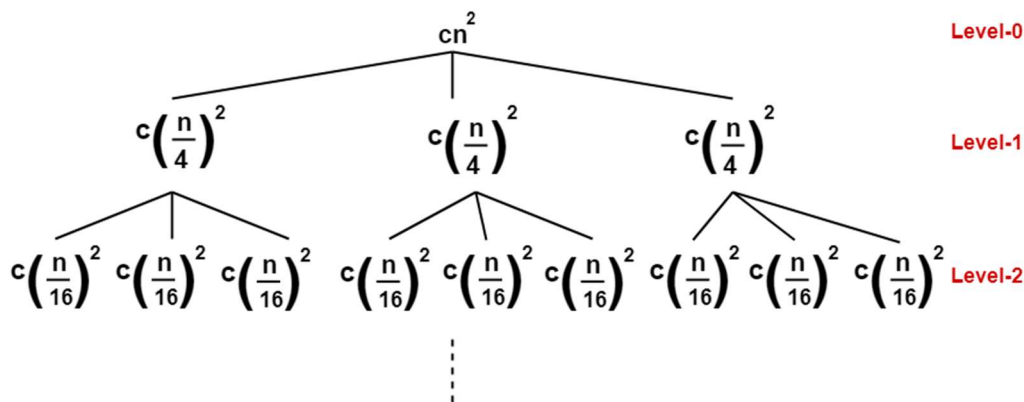
Solve the following recurrence relation using recursion tree method:

$$T(n) = 3T(n/4) + cn^2$$

Solution-

Step-01:

Draw a recursion tree based on the given recurrence relation:



(Here, we have directly drawn a recursion tree representing the cost of sub problems)

Step-02:

Determine cost of each level:

- Cost of level-0 = cn^2
- Cost of level-1 = $c(n/4)^2 + c(n/4)^2 + c(n/4)^2 = (3/16)cn^2$
- Cost of level-2 = $c(n/16)^2 \times 9 = (9/16^2)cn^2$

Step-03:

Determine total number of levels in the recursion tree:

- Size of problem at level-0 = $n/4^0$
- Size of sub-problem at level-1 = $n/4^1$
- Size of sub-problem at level-2 = $n/4^2$

Continuing in similar manner, we have,

Size of sub-problem at level-i = $n/4^i$

Suppose at level-x (last level), size of sub-problem becomes 1. Then,

$$n/4^x = 1$$

$$4^x = n$$

$$x = \log_4 n$$

$$\therefore \text{Total number of levels in the recursion tree} = \log_4 n + 1$$

Step-04:

Determine number of nodes in the last level:

- Level-0 has 3^0 nodes i.e., 1 node
- Level-1 has 3^1 nodes i.e., 3 nodes
- Level-2 has 3^2 nodes i.e., 9 nodes

Continuing in similar manner, we have,

Last level (i.e., level - $\log_4 n$) has $3^{\log_4 n} = n^{\log_4 3}$ nodes.

Step-05:

Determine cost of last level:

$$\text{Cost of last level} = n^{\log_4 3} * T(1) = \theta(n^{\log_4 3})$$

Step-06:

Add costs of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation:

$$T(n) = \underbrace{\left\{ cn^2 + \frac{3}{16} cn^2 + \frac{9}{(16)^2} cn^2 + \dots \right\}}_{\text{For } \log_4 n \text{ levels}} + \theta(n^{\log_4 3})$$

$$= cn^2 \{ 1 + (3/16) + (3/16)^2 + \dots \} + \theta(n^{\log_4 3})$$

Now, $\{ 1 + (3/16) + (3/16)^2 + \dots + (3/16)^{i-1} \}$ forms Geometric progression, where $i = \log_4 n$.

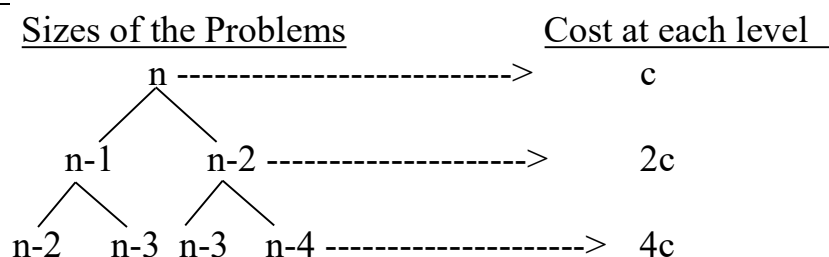
On solving, we get,

$$\begin{aligned}
&= (16/13)cn^2 (1 - (3/16)^{\log_4 n}) + \theta(n^{\log_4 3}) \\
&= (16/13)cn^2 - (16/13)cn^2 (3/16)^{\log_4 n} + \theta(n^{\log_4 3}) \\
&= (16/13)cn^2 - (16/13)cn^2 ((3)^{\log_4 n} / (16)^{\log_4 n}) + \theta(n^{\log_4 3}) \\
&= (16/13)cn^2 - (16/13)cn^2 (n^{\log_4 3} / n^{\log_4 16}) + \theta(n^{\log_4 3}) \\
&= (16/13)cn^2 - (16/13)cn^2 (n^{\log_4 3} / n^2) + \theta(n^{\log_4 3}) \\
&= (16/13)cn^2 - (16/13)c (n^{\log_4 3}) + \theta(n^{\log_4 3}) \\
&= O(n^2)
\end{aligned}$$

Problem-4:

$$T(n) = T(n-1) + T(n-2) + c$$

Solution:



The size of larger subproblem at last level (level x) is $n-x=1$. So, $x=n-1$.

The last level has 2^x nodes. So, the cost of last level is $2^x c$.

The total cost of all levels = $(c + 2c + 2^2 c + \dots + 2^{x-1} c) + 2^x c$

$$= c(1 + 2 + 2^2 + \dots + 2^x)$$

$$= c(2^{x+1} - 1)$$

$$= O(2^{x+1}) = O(2^n).$$

Master Theorem for solving recurrence relations of divide-and-conquer: -

Let $T(n) = aT(n/b) + f(n)$, then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$, for some constant $\epsilon > 0$ (i.e., if $f(n) < n^{\log_b a}$), then $T(n) = \theta(n^{\log_b a})$.

2. (a) If $f(n) = \theta(n^{\log_b a})$, then $T(n) = \theta(n^{\log_b a} \lg n)$.

(b) If $f(n) = \theta(n^{\log_b a} \lg^k n)$, where $k \geq 0$, then $T(n) = \theta(n^{\log_b a} \lg^{k+1} n)$.

3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$ (i.e., if $f(n) > n^{\log_b a}$), and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \theta(f(n))$.

Examples:

(1).

$$T(n) = 9T(n/3) + n.$$

For this recurrence, we have $a = 9$, $b = 3$, $f(n) = n$, and thus we have that $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$. Since $f(n) = O(n^{\log_3 9 - \epsilon})$, where $\epsilon = 1$, we can apply case 1 of the master theorem and conclude that the solution is $T(n) = \Theta(n^2)$.

(2).

$$T(n) = T(2n/3) + 1,$$

in which $a = 1$, $b = 3/2$, $f(n) = 1$, and $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$. Case 2a applies, since $f(n) = \Theta(n^{\log_b a}) = \Theta(1)$, and thus the solution to the recurrence is $T(n) = \Theta(\lg n)$.

(3)

$$T(n) = 3T(n/4) + n \lg n,$$

we have $a = 3$, $b = 4$, $f(n) = n \lg n$, and $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$. Since $f(n) = \Omega(n^{\log_4 3 + \epsilon})$, where $\epsilon \approx 0.2$, case 3 applies if we can show that the regularity condition holds for $f(n)$. For sufficiently large n , we have that $af(n/b) = 3(n/4) \lg(n/4) \leq (3/4)n \lg n = cf(n)$ for $c = 3/4$. Consequently, by case 3, the solution to the recurrence is $T(n) = \Theta(n \lg n)$.

(4)

$$T(n) = 2T(n/2) + n \lg n$$

even though it appears to have the proper form: $a = 2$, $b = 2$, $f(n) = n \lg n$, and $n^{\log_b a} = n$. You might mistakenly think that case 3 should apply, since

$f(n) = n \lg n$ is asymptotically larger than $n^{\log_b a} = n$. The problem is that it is not *polynomially* larger. The ratio $f(n)/n^{\log_b a} = (n \lg n)/n = \lg n$ is asymptotically less than n^ϵ for any positive constant ϵ .

Apply case 2b of master theorem to solve the above problem.

$$T(n) = \Theta(n \lg^2 n).$$

Binary search: -

- Let $a[1:n]$ be a list of elements that are sorted in ascending order.
- We have to determine whether a given element x is present in the list or not.
- If x is present, we have to return its position, else return 0.

Iterative algorithm: -

Algorithm BinSearch(a, n, x)

// Given an array $a[1:n]$ of elements in nondecreasing order, $n > 0$, determine

// whether x is present, and if so, return j such that $x = a[j]$; else return 0.

```
{
    low := 1; high := n;
    while (low ≤ high) do // If there is at least one element in the given part of array
    {
        mid := ⌊(low + high)/2⌋;
        if ( $x < a[mid]$ ) then high := mid - 1;
        else if ( $x > a[mid]$ ) then low := mid + 1;
        else return mid;
    }
    return 0;
}
```

Recursive algorithm: -

Algorithm RBinSearch($a, low, high, x$)

```
{
    // Given an array a[low: high] of elements in nondecreasing
    // order, 1 < low < high, determine whether x is present, and
    // if so, return j such that x = a[j]; else return 0.
```

```
    if (low > high) then // If there are no elements in the given part of array
        return 0;
```

```
    else
```

```
    {
```

```
        // Reduce the problem into a smaller subproblem.
```

```
        mid := ⌊(low + high)/2⌋;
```

```
        if ( $x = a[mid]$ ) then
```

```
            return mid;
```

```
        else if ( $x < a[mid]$ ) then
```

```

        return RBinSearch(a, low, mid-1, x);
    else
        return RBinSearch(a, mid+1, high, x);
    }
}

```

→ This algorithm is initially invoked as RBinsearch(a,1, n, x).

Time Complexity:

→ The main problem is divided into one sub-problem in constant time.

→ The answer to the new sub-problem is also the answer to the original problem.

So, there is no need for combining.

→ The running-time of this algorithm can be characterized as follows:

$$T(n) = \begin{cases} c, & \text{if } n = 1 \\ T(n/2) + c, & \text{otherwise} \end{cases}$$

Solving by using substitution method,

$$T(n) = T(n/2) + c$$

$$= [T(n/4) + c] + c = T(n/4) + 2c$$

$$= [T(n/8) + c] + 2c = T(n/8) + 3c = T(n/2^3) + 3c$$

After applying this substitution k times,

$$T(n) = T(n/2^k) + kc$$

To terminate this substitution process, we switch to the closed form $T(1)=c$, which happens when $2^k = n$ which implies $k = \log_2 n$

$$\text{So, } T(n) = T(1) + c \log_2 n$$

$$T(n) = c + c \log_2 n$$

$$= O(\log n) \text{ --- Worst-case time complexity}$$

The best-case time complexity is $O(1)$.

Merge sort: -

→ Given a sequence of n elements $a[1:n]$, we divide the given set of elements into two subsets $a[1:n/2]$ and $a[n/2+1:n]$.

→ Each subset is individually sorted, and the resulting sorted sequences are merged to produce a single sorted sequence of n elements.

Recursive algorithm for merge sort:

Algorithm MergeSort(low, high)

```
{
    // a[low : high] is a global array to be sorted.

    if (low < high) then // If there are more than one element in the given part of array
    {
        mid := [(low + high) / 2];
        MergeSort(low, mid);           // Solve sub-problem 1
        MergeSort(mid + 1, high);      // Solve sub-problem 2
        Merge(low, mid, high);         // Combine the solutions of sub-problems
    }
}
```

Algorithm Merge(low, mid, high)

```
{
    // a[low : high] is a global array containing two sorted subsets in a[low : mid] and
    // in a[mid + 1 : high]. The goal is to merge these two sets into a single set residing
    // in a[low : high].
    // b[ ] is an auxiliary(temporary) global array.

    i := low;
    h := low;           // h is the indexing variable for the first part of the array a[low : high].
    j := mid + 1;       // j is the indexing variable for the second part of the array a[low : high].
    while ((h ≤ mid) and (j ≤ high)) do
    {
        if (a[h] ≤ a[j]) then
        {
            b[i] := a[h];    // copy the elements of a into b.
            h := h + 1;
        }
        else
        {
            b[i] := a[j];    // copy the elements of a into b.
            j := j + 1;
        }
        i := i + 1;
    } //end of while
}
```

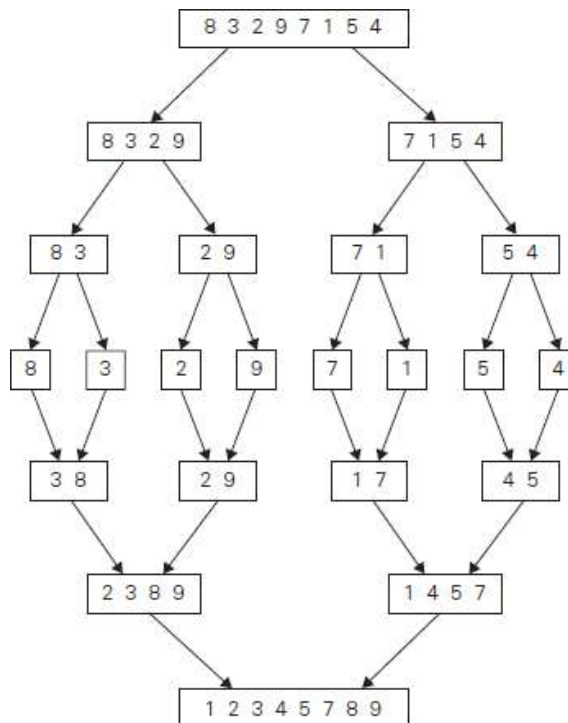
```

if (h>mid) then
{
    for k := j to high do
    {
        b[i]:=a[k]; // copy the remaining elements of second part of a into b.
        i:=i+1;
    }
}
else
{
    for k:=h to mid do
    {
        b[i] := a[k]; // copy the remaining elements of first part of a into b.
        i:=i+1;
    }
}
for k: = low to high do
    a[k]:=b[k]; // copy the elements of b into a.
} // end of algorithm.

```

→ MergeSort algorithm is initially invoked as MergeSort(1, n).

Example: Trace the MergeSort on the data: 8,3,2,9,7,1,5,4.



Time Complexity:

→ The merge-sort problem of size n is divided into two sub-problems of size $n/2$ each.

This division is done in constant time by the statement $mid := \lfloor (low + high)/2 \rfloor$;

→ The solutions to the sub-problems are merged in linear time.

So,

$$T(n) = \begin{cases} c, & \text{if } n = 1 \\ 2T(n/2) + cn, & \text{otherwise} \end{cases}$$

Solving by using substitution method,

$$T(n) = 2T(n/2) + cn$$

$$= 2[2T(n/4) + cn/2] + cn = 4T(n/4) + cn + cn = 4T(n/4) + 2cn$$

$$= 4[2T(n/8) + cn/4] + 2cn = 8T(n/8) + 3cn = 2^3T(n/2^3) + 3c$$

After applying this substitution k times,

$$T(n) = 2^kT(n/2^k) + kcn$$

To terminate this substitution process, we switch to the closed form $T(1)=c$, which happens when $2^k = n$ which implies $k = \log_2 n$.

So,

$$T(n) = nT(1) + c \cdot \log_2 n \cdot n$$

$$T(n) = cn + cn \log_2 n$$

$$T(n) = O(n \log n)$$

This is the best-case, worst-case and average-case time complexity.

Quick sort: -

→ In quicksort, the division of $a[1:n]$ into two subarrays is made so that, the sorted subarrays do not need to be merged later.

→ This is accomplished by picking some element of $a[]$, say t , which is called *pivot(or, portioning element)*, and then reordering the other elements such that all the elements which are less than or equal to t are placed before t and all the elements which are greater than t are placed after t in the array $a[1:n]$.

This rearrangement is referred to as *partitioning*.

→ Although there are many choices for the pivot, here we assume that the *first element in the array acts as pivot*, for convenience.

→ Partition of a set of elements S about the pivot t produces two disjoint subsets S_1 and S_2 , where

$$S_1 = \{x \in S - \{t\} \mid x \leq t\} \text{ and}$$

$$S_2 = \{x \in S - \{t\} \mid x > t\}.$$

→ Obviously, after a partition is achieved, pivot element t will be in its final position in the sorted array, and we can continue sorting the two subarrays to the left and to the right of t independently.

→ **Note:** The difference between mergesort and quicksort is as follows:

In mergesort, the division of the problem into two subproblems is immediate and the entire work happens in combining their solutions; but in quicksort, the entire work happens in the division stage, with no work required to combine the solutions to the subproblems.

Recursive algorithm for quick sort:

Algorithm QuickSort(low, high)

```
{
    // Sorts the elements a[low], ..., a[high] which reside in the global
    // array a[1 : n] into ascending order.
    // a[n+1] is considered which must be greater than or equal to all elements in a[1:n].

    if (low < high) then    // If there are more than one element
    {
        j := Partition(a, low, high+1); // j is the final position of the partitioning element.
        QuickSort(low, j - 1); // Solve subproblem 1
        QuickSort(j+1, high); // Solve subproblem 2
        // There is no need for combining solutions.
    }
}
```

Algorithm Partition(a, l, h)

```
{
    // a[l] is considered pivot
    t := a[l];
    lp := l+1;
    rp := h-1;
    while (lp ≤ rp) do
    {
```

```

while(a[lp]≤t) do lp:=lp+1;
while(a[rp]>t) do rp:=rp-1;
if(lp<rp) then
{
    // swap a[lp] and a[rp]
    temp:= a[lp];
    a[lp]:=a[rp];
    a[rp]:=temp;
}
}
a[l]:=a[rp];
a[rp]:=t;
return rp;
}

```

→Initially, QuickSort is invoked as QuickSort(1,n).

Example: Trace the QuickSort algorithm on the following data:
65, 70, 75, 80, 60, 55, 50, 45.

Index:	1	2	3	4	5	6	7	8	9
Elements:	<u>65</u>	^{lp} 70	75	80	60	55	50	^{rp} 45	∞
	<u>65</u>	^{lp} 45	75	80	60	55	50	^{rp} 70	∞
	<u>65</u>	45	^{lp} 75	80	60	55	^{rp} 50	70	∞
	<u>65</u>	45	^{lp} 50	80	60	55	^{rp} 75	70	∞
	<u>65</u>	45	50	^{lp} 80	60	^{rp} 55	75	70	∞
	<u>65</u>	45	50	^{lp} 55	60	^{rp} 80	75	70	∞
	<u>65</u>	45	50	55	^{rp} 60	^{lp} 80	75	70	∞
	<u>60</u>	45	50	55	65	80	75	70	∞
	<u>60</u>	^{lp} 45	50	^{rp} 55					∞
				^{rp}	^{lp}				

<u>60</u>	45	50	55					∞
<u>55</u>	45	50	60					∞
	<i>lp</i>	<i>rp</i>						
<u>55</u>	45	50						∞
		<i>rp</i>	<i>lp</i>					
<u>55</u>	45	50						∞
<u>50</u>	45	55						∞
<u>50</u>	<i>lp, rp</i>							∞
	45							
	<i>rp</i>	<i>lp</i>						
<u>50</u>	45							∞
45	50							∞
						<i>lp</i>	<i>rp</i>	
					<u>80</u>	75	70	∞
					<u>80</u>	75	<i>rp</i> 70	<i>lp</i> ∞
					<u>70</u>	75	80	∞
						<i>lp, rp</i> 75		∞
					<u>70</u>	<i>rp</i> 75		∞
					<u>70</u>	<i>lp</i> 75		∞
					70	75		∞
45	50	55	60	65	70	75	80	

So, the sorted data is: 45,50, 55, 60, 65, 70, 75, 80.

Time Complexity:

Quick Sort partitions the given set of elements S into two subsets S1 and S2 around pivot element t . That means, $S=S1 \cup \{t\} \cup S2$.

If $|S| = n$ and $|S1| = i$, then $|S2| = n-i-1$. So, $T(n) = T(i) + T(n-i-1) + f(n)$

$T(n) = T(i) + T(n-i-1) + cn$; since the partitioning is done in linear time.

(i) Worst-Case Analysis:

This occurs when the elements are in either ascending order or descending order.

→ Consider that the elements are in ascending order. Then,

$$|S1| = 0 \text{ and } |S2| = n-1.$$

So,

$$T(n) = T(0) + T(n-1) + cn$$

$$T(n) = c + T(n-1) + cn$$

Neglecting the insignificant constant term,

$$T(n) = T(n-1) + cn$$

Solving using substitution method,

$$T(n) = T(n-2) + cn + c(n-1)$$

$$T(n) = T(n-3) + cn + c(n-1) + c(n-2)$$

After k substitutions,

$$T(n) = T(n-k) + cn + c(n-1) + c(n-2) + \dots + c(n-(k-1))$$

$$= T(n-k) + c(n+(n-1)+(n-2)+\dots+(n-(k-1)))$$

Assume $n=k$,

$$T(n) = T(0) + c(k+(k-1)+(k-2)+\dots+1)$$

$$= c + c(1+2+\dots+k)$$

$$= c + c(k(k+1)/2)$$

$$= c + c(n(n+1)/2) \quad T(n) = O(n^2).$$

(ii) Best-case Analysis:

This occurs when the pivot element occupies middle position after partitioning. In this case, $|S1| = n/2$ and $|S2| = n/2$.

So,

$$T(n) = 2T(n/2) + cn$$

Solving this using substitution method,

$$T(n) = 2T(n/2) + cn$$

$$= 2[2T(n/4) + cn/2] + cn = 4T(n/4) + cn + cn = 4T(n/4) + 2cn$$

$$= 4[2T(n/8) + cn/4] + 2cn = 8T(n/8) + 3cn = 2^3T(n/2^3) + 3c$$

After applying this substitution k times,

$$T(n) = 2^k T(n/2^k) + kcn$$

To terminate this substitution process, we switch to the closed form $T(1)=c$, which

happens when $2^k = n$ which implies $k = \log_2 n$.

So,

$$T(n) = nT(1) + c \cdot \log_2 n \cdot n$$

$$T(n) = cn + cn \log_2 n$$

$$T(n) = O(n \log n).$$

(iii) Average-Case Analysis: (Using Probability Analysis)

→ After partitioning is performed, the pivot element may occupy any position from 1 to n in the array. So, S_1 can have any size from 0 to $n-1$ with equal probability of $1/n$. Similarly, S_2 also can have any size from 0 to $n-1$ with equal probability of $1/n$.

And we know that $T(n) = T(i) + T(n-i-1) + cn$.

→ So, the expected or average value of $T(i)$ is $\frac{\sum_{j=0}^{n-1} T(j)}{n}$.

Similarly, the expected or average value of $T(n-i-1)$ is also $\frac{\sum_{j=0}^{n-1} T(j)}{n}$.

Now, the equation $T(n) = T(i) + T(n-i-1) + cn$, becomes

$$T(n) = \frac{2 \sum_{j=0}^{n-1} T(j)}{n} + cn \quad \text{----- (1)}$$

If equation (1) is multiplied by n , it becomes

$$nT(n) = 2 \sum_{j=0}^{n-1} T(j) + cn^2 \quad \text{----- (2)}$$

From (2) we can get

$$(n-1)T(n-1) = 2 \sum_{j=0}^{n-2} T(j) + c(n-1)^2 \quad \text{----- (3)}$$

Subtracting (3) from (2),

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + c(n^2 - (n-1)^2)$$

$$nT(n) = (n+1) T(n-1) + 2nc - c$$

Dropping insignificant term $-c$, we get

$$nT(n) = (n+1) T(n-1) + 2nc \quad \text{----- (4)}$$

Dividing (4) by $n(n+1)$, we get

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2c}{n+1} \quad \text{----- (5)}$$

Solving the above equation using substitution method,

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2c}{n+1}$$

$$\frac{T(n)}{n+1} = \left(\frac{T(n-2)}{n-1} + \frac{2c}{n} \right) + \frac{2c}{n+1}$$

$$\frac{T(n)}{n+1} = \frac{T(n-3)}{n-2} + \frac{2c}{n-1} + \frac{2c}{n} + \frac{2c}{n+1}$$

After k substitutions,

$$\frac{T(n)}{n+1} = \frac{T(n-k)}{n-k+1} + 2c \left(\frac{1}{n-k+2} + \frac{1}{n-k+3} + \dots + \frac{1}{n+1} \right)$$

When $n=k$,

$$\frac{T(n)}{n+1} = \frac{T(0)}{1} + 2c \left(\frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n+1} \right)$$

$$\frac{T(n)}{n+1} = T(0) + 2c \left(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n+1} - 1 \right)$$

Since $\sum_{i=1}^n \frac{1}{i} \approx \log_e n + \gamma$, where $\gamma = 0.577$ (Euler's constant),

$$\begin{aligned} \frac{T(n)}{n+1} &= c + 2c(\log_e(n+1) + \gamma - 1) \\ &= c + 2c\log_e(n+1) + 2c(\gamma - 1) \\ &= c + 2c\log_e(n+1) \end{aligned}$$

$$\begin{aligned} T(n) &= (n+1)(c + 2c\log_e(n+1)) \\ &= c(n+1) + 2c(n+1)\log_e(n+1) \\ &= c(n+1) + 2cn\log_e(n+1) + 2c\log_e(n+1) \end{aligned}$$

$$T(n) = O(n \log n)$$

Matrix multiplication: -

Suppose we are given two square matrices A and B of order $n \times n$ each, and we wish to compute their product $C = A * B$, which is also an $n \times n$ matrix whose $[i,j]^{\text{th}}$ element is found by taking the elements in the i^{th} row of A and j^{th} column of B and multiplying them. So, we get,

$$C[i, j] = \sum_{1 \leq k \leq n} A[i, k] * B[k, j]$$

To compute any element $C[i,j]$ using this formula, we need n multiplications. As the matrix C has n^2 elements, the time for the resulting matrix multiplication algorithm is $O(n^3)$.

Algorithm MatrixMul(A, B, n)

```
{
    for i:=1 to n do
    {
        for j:=1 to n do
        {
            C[i,j]:=0;
            for k:=1 to n do
            {
                C[i,j]:= C[i,j]+ A[i,k]*B[k,j];
            }
        }
    }
}
```

Matrix multiplication using divide & conquer strategy: -

Suppose we want to perform $A_{n \times n} * B_{n \times n} = C_{n \times n}$.

We assume that n is a power of 2.

→ We divide each matrix A and B into four quadrants (or, submatrices) of order $n/2 \times n/2$ each.

→ Then each matrix A and B can be viewed as a 2×2 matrix, whose elements are $n/2 \times n/2$ matrices. Then the product $A * B$ can be computed like the product of two 2×2 matrices. So, we can rewrite the equation $C = A * B$ as:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

where, each A_{ij} , B_{ij} and C_{ij} is an $n/2 \times n/2$ matrix.

Now, the submatrices of C can be computed as follows:

$$C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$$

$$C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$$

$$C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$$

$$C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$$

→ Each of these four equations specifies two multiplications of $n/2 \times n/2$ matrices and an addition of their $n/2 \times n/2$ products. So, total 8 multiplications and 4 additions are required.

→ We can use these equations to create a straightforward, recursive, divide-and-conquer algorithm. This algorithm will perform 8 recursive calls and will continue applying itself to smaller-sized submatrices until n becomes suitably small (i.e., $n=2$) in which case the product is computed directly (in constant time).

→ Since two $n/2 \times n/2$ matrices can be added in cn^2 time for some constant c , 4 additions of this similar kind can also be performed in cn^2 time.

→ So, the overall computing time $T(n)$ of the resulting divide-and-conquer algorithm is given by the recurrence:

$$T(n) = \begin{cases} c, & \text{if } n \leq 2 \\ 8T\left(\frac{n}{2}\right) + cn^2, & \text{if } n > 2 \end{cases}$$

Solving the recurrence relation using substitution method,

$$\begin{aligned} T(n) &= 8T(n/2) + cn^2 \\ &= 8[8T(n/2^2) + c(n/2)^2] + cn^2 = 8^2T(n/2^2) + 2cn^2 + cn^2 = 8^2T(n/2^2) + 3cn^2 \\ &= 8^2[8T(n/2^3) + c(n/2^2)^2] + 3cn^2 = 8^3T(n/2^3) + 4cn^2 + 3cn^2 = 8^3T(n/2^3) + 7cn^2 \\ &= 8^3T(n/2^3) + (2^3 - 1)cn^2 \end{aligned}$$

After k substitutions,

$$\begin{aligned} T(n) &= 8^k T(n/2^k) + (2^k - 1)cn^2 \\ &= (2^3)^k T(n/2^k) + (2^k - 1)cn^2 \\ &= (2^k)^3 T(n/2^k) + (2^k - 1)cn^2 \end{aligned}$$

Assume $n = 2^k$ which implies $k = \log_2 n$

Now,

$$\begin{aligned} T(n) &= n^3 T(1) + (n-1)cn^2 \\ &= cn^3 + cn^3 - cn^2 \\ &= O(n^3) \end{aligned}$$

→ Even though we used divide and conquer strategy, no improvement has been made in the time complexity over the straightforward matrix multiplication algorithm.

→ Since, matrix multiplications are more expensive than matrix additions ($O(n^3)$ versus $O(n^2)$), we can reformulate the equations for C_{ij} in order to have fewer multiplications and more additions. Volker Strassen has discovered a way to compute the C_{ij} 's using only 7 multiplications and 18 additions or subtractions.

Strassen's matrix multiplication: -

→ Given two matrices A and B of order $n \times n$ each, we need^{to} compute their product $C = A * B$, which is also an $n \times n$ matrix. Each of the matrices A and B is divided into four quadrants (or, submatrices) of order $n/2 \times n/2$ each.

So, we can rewrite the equation $C = A * B$ as:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

In this method, the following 7 equations are used to obtain 7 matrices of order $n/2 \times n/2$ each, as follows:

$$P = (A_{11} + A_{22}) * (B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22}) * B_{11}$$

$$R = A_{11} * (B_{12} - B_{22})$$

$$S = A_{22} * (B_{21} - B_{11})$$

$$T = (A_{11} + A_{12}) * B_{22}$$

$$U = (A_{21} - A_{11}) * (B_{11} + B_{12})$$

$$V = (A_{12} - A_{22}) * (B_{21} + B_{22})$$

Now, the submatrices of C matrix can be obtained as follows:

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

→ As can be seen above, the 7 matrices P, Q, R, S, T, U, and V can be computed using total 7 matrix multiplications and 10 matrix additions or subtractions. The C_{ij} 's require an additional 8 additions or subtractions.

→ So, the Strassen's matrix multiplication uses total 7 multiplications and 18 additions or subtractions. The 18 additions can be performed in cn^2 time.

The resulting recurrence relation for $T(n)$ is:

$$T(n) = \begin{cases} c, & \text{if } n \leq 2 \\ 7T\left(\frac{n}{2}\right) + cn^2, & \text{if } n > 2 \end{cases}$$

Solving the recurrence relation using substitution method,

$$\begin{aligned} T(n) &= 7T(n/2) + cn^2 \\ &= 7[7T(n/2^2) + c(n/2)^2] + cn^2 = 7^2T(n/2^2) + \left(\frac{7}{4}\right)cn^2 + cn^2 \\ &= 7^3T(n/2^3) + \left(\frac{7}{4}\right)^2cn^2 + \left(\frac{7}{4}\right)cn^2 + cn^2 \end{aligned}$$

After k substitutions,

$$\begin{aligned} T(n) &= 7^kT(n/2^k) + cn^2\left(1 + \left(\frac{7}{4}\right) + \left(\frac{7}{4}\right)^2 + \dots + \left(\frac{7}{4}\right)^{k-1}\right) \\ &= 7^kT(n/2^k) + cn^2\left(\frac{\left(\frac{7}{4}\right)^k - 1}{\left(\frac{7}{4}\right) - 1}\right) \end{aligned}$$

Assume $n = 2^k$ which implies $k = \log_2 n$

Now,

$$\begin{aligned} T(n) &= 7^{\log_2 n}T(1) + cn^2\left(\frac{\left(\frac{7}{4}\right)^{\log_2 n} - 1}{\left(\frac{7}{4}\right) - 1}\right) \\ &= n^{\log_2 7}T(1) + cn^2\left(\frac{\left(\frac{7^{\log_2 n}}{4^{\log_2 n}}\right) - 1}{\left(\frac{7}{4}\right) - 1}\right) \\ &= cn^{\log_2 7} + cn^2\left(\frac{\left(\frac{n^{\log_2 7}}{n^{\log_2 4}}\right) - 1}{\left(\frac{7}{4}\right) - 1}\right) \\ &= cn^{\log_2 7} + cn^2\left(\frac{\left(\frac{n^{\log_2 7}}{n^2}\right) - 1}{\left(\frac{7}{4}\right) - 1}\right) \\ &= cn^{\log_2 7} + c\left(\frac{n^{\log_2 7} - n^2}{\left(\frac{7}{4}\right) - 1}\right) \\ &= cn^{\log_2 7} + c(n^{\log_2 7} - n^2) \\ &= cn^{\log_2 7} + cn^{\log_2 7} - cn^2 \end{aligned}$$

$$T(n) = O(n^{\log_2 7}) = O(n^{2.81})$$

Example: Multiply the following matrices using Strassen's algorithm:

$$A = \begin{bmatrix} 1 & 2 & 2 & 1 \\ 2 & 1 & 2 & 1 \\ 1 & 2 & 1 & 2 \\ 1 & 1 & 2 & 2 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 2 & 1 & 2 & 2 \\ 1 & 1 & 1 & 2 \\ 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 2 \end{bmatrix}$$

Sol:

We want to perform $C = A * B$.

Divide the given matrices into $n/2 \times n/2$ submatrices each.

So, each of the given 4×4 matrices is divided into four 2×2 submatrices as follows:

$$A_{11} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \quad A_{12} = \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} \quad A_{21} = \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} \quad A_{22} = \begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix}$$

$$B_{11} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \quad B_{12} = \begin{bmatrix} 2 & 2 \\ 1 & 2 \end{bmatrix} \quad B_{21} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \quad B_{22} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$P = (A_{11} + A_{22}) * (B_{11} + B_{22})$$

$$= \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix} * \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 4 \\ 4 & 3 \end{bmatrix} * \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$$

$$= \begin{bmatrix} 14 & 16 \\ 18 & 17 \end{bmatrix}$$

$$Q = (A_{21} + A_{22}) * B_{11}$$

$$= \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix} * \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 4 \\ 3 & 3 \end{bmatrix} * \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 8 & 6 \\ 9 & 6 \end{bmatrix}$$

$$R = A_{11} * (B_{12} - B_{22})$$

$$= \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} * \begin{bmatrix} 2 & 2 \\ 1 & 2 \end{bmatrix} - \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$$

$$S = A_{22} * (B_{21} - B_{11})$$

$$= \begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix} * \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} - \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix} * \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 2 \\ 0 & 2 \end{bmatrix}$$

$$T = (A_{11} + A_{12}) * B_{22}$$

$$= \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} + \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 3 \\ 4 & 2 \end{bmatrix} * \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 6 & 9 \\ 6 & 8 \end{bmatrix}$$

$$U = (A_{21} - A_{11}) * (B_{11} + B_{12})$$

$$= \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} - \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} * \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 2 & 2 \\ 1 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 \\ -1 & 0 \end{bmatrix} * \begin{bmatrix} 4 & 3 \\ 2 & 3 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 \\ -4 & -3 \end{bmatrix}$$

$$V = (A_{12} - A_{22}) * (B_{21} + B_{22})$$

$$= \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} - \begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix} * \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & -1 \\ 0 & -1 \end{bmatrix} * \begin{bmatrix} 3 & 2 \\ 2 & 4 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & -2 \\ -2 & -4 \end{bmatrix}$$

Now calculating the submatrices of resultant matrix C:

$$C_{11} = P + S - T + V$$

$$\begin{bmatrix} 14 & 16 \\ 18 & 17 \end{bmatrix} + \begin{bmatrix} 0 & 2 \\ 0 & 2 \end{bmatrix} - \begin{bmatrix} 6 & 9 \\ 6 & 8 \end{bmatrix} + \begin{bmatrix} 1 & -2 \\ -2 & -4 \end{bmatrix}$$

$$= \begin{bmatrix} 9 & 7 \\ 10 & 7 \end{bmatrix}$$

$$C_{12} = R + T$$

$$= \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} + \begin{bmatrix} 6 & 9 \\ 6 & 8 \end{bmatrix}$$

$$= \begin{bmatrix} 7 & 10 \\ 8 & 10 \end{bmatrix}$$

$$C_{21}=Q+S$$

$$= \begin{bmatrix} 8 & 6 \\ 9 & 6 \end{bmatrix} + \begin{bmatrix} 0 & 2 \\ 0 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 8 & 8 \\ 9 & 8 \end{bmatrix}$$

$$C_{22}=P+R-Q+U$$

$$= \begin{bmatrix} 14 & 16 \\ 18 & 17 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} - \begin{bmatrix} 8 & 6 \\ 9 & 6 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ -4 & -3 \end{bmatrix}$$

$$= \begin{bmatrix} 7 & 11 \\ 7 & 11 \end{bmatrix}$$

$$\text{The resultant matrix, } C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} 9 & 7 & 7 & 10 \\ 10 & 7 & 8 & 10 \\ 8 & 8 & 7 & 11 \\ 9 & 8 & 7 & 10 \end{bmatrix}$$